*BRACT's*

**Vishwakarma Institute of Technology, Pune**

**Practical Implementation Sheet**

Department: CSE-AI Semester: 5 Academic Year: 2026-27

# Practical No: 3

Division: C Batch: 2

Roll No: 3090358 PRN No: 12410037

Name of Student: Radhyeshyam Rameshwar Lahoti

**Subject Name: Deep Learning**

## Problem Statement:

Implement forward propagation and backpropagation using TensorFlow/Keras. Analyze the effect of different learning rates and the number of epochs on model performance.

## Algorithm :

**Fashion MNIST Dataset**

The Fashion MNIST dataset is a benchmark dataset used for image classification tasks. It contains 70,000 grayscale images of fashion products belonging to 10 different classes.

- Training Samples: 60,000
- Testing Samples: 10,000
- Image Size: 28 × 28 pixels

The dataset contains the following classes:

- T-shirt/Top
- Trouser
- Pullover
- Dress
- Coat
- Sandal
- Shirt
- Sneaker
- Bag
- Ankle Boot

Each image contains 784 pixel values (28 × 28), and the target variable represents the clothing category.

**Data Preprocessing**
Before training the neural network, the dataset is preprocessed:

- Reshaping: Each 28 × 28 image is flattened into a 784-dimensional vector to serve as input to the neural network.
- Normalization: Pixel values are scaled from 0–255 to 0–1 by dividing each pixel value by 255.0. This improves training stability and convergence.
- Validation Split: During training, 20% of the training data is used as validation data to monitor model performance.

**Artificial Neural Network (ANN) Model**
The model uses:

- Input Layer: 784 neurons (one for each pixel)
- Hidden Layer 1: 128 neurons
- Hidden Layer 2: 64 neurons
- Output Layer: 10 neurons (one for each class)
- Activation Function (Hidden Layers): ReLU (Rectified Linear Unit)
- Output Activation: Softmax
- Optimizer: Adam
- Loss Function: Sparse Categorical Crossentropy
- Batch Size: 128
- Learning Rates Tested: 0.1, 0.01, 0.001
- Epochs Tested: 5, 10, 20

**ReLU Activation Function**
ReLU introduces non-linearity into the network and is defined as:

$$f(x) = \max(0, x)$$

It outputs the input directly if it is positive; otherwise, it outputs zero. ReLU helps overcome the vanishing gradient problem and speeds up model training.

**Softmax Activation Function**
The Softmax activation function is used in the output layer for multi-class classification.

$$P(y_i) = \frac{e^{z_i}}{\sum e^{z_j}}$$

It converts the output values into probabilities, where the sum of all probabilities equals 1. The class with the highest probability is selected as the predicted class.

**Adam Optimizer**
Adam (Adaptive Moment Estimation) is an optimization algorithm used to efficiently update the network weights during training. It combines the advantages of Momentum and RMSProp, resulting in faster convergence and better performance.

**Training Process**

1. Fashion MNIST images are loaded and preprocessed.
2. Images are reshaped into 784-dimensional vectors and normalized.
3. The ANN model is initialized with two hidden layers.
4. Forward propagation computes the output probabilities.
5. The Sparse Categorical Crossentropy loss is calculated.
6. Backpropagation updates the weights using the Adam optimizer.
7. The process repeats for the specified number of epochs until the model converges.

**Forward Propagation**

Forward propagation is the process of passing input data through the neural network from the input layer to the output layer to generate predictions. The input is multiplied by weights, biases are added, activation functions (ReLU and Softmax) are applied, and the final output is produced.

**Backpropagation**

Backpropagation is the process of updating the network's weights and biases by calculating the prediction error and propagating it backward through the network. The Adam optimizer adjusts the weights to minimize the loss and improve the model's accuracy.

**Hyperparameter Analysis**

- **Learning Rate Analysis**
  Different learning rates (0.1, 0.01, and 0.001) are tested to observe their effect on model accuracy. The learning rate controls how much the model updates its weights during each iteration.

- **Epoch Analysis**
  The model is trained using 5, 10, and 20 epochs to study how increasing the number of training iterations affects classification accuracy and convergence.

**Performance Evaluation**

**Accuracy**

Accuracy measures the percentage of images correctly classified.

$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Predictions}}$$

A higher accuracy indicates better classification performance.

**Confusion Matrix**

A confusion matrix shows the number of correct and incorrect predictions for each clothing class. The diagonal elements represent correctly classified images, while the off-diagonal elements represent misclassifications.

**Classification Report**

The classification report includes:

- Precision: The proportion of correctly predicted samples among all samples predicted for a particular class.
- Recall: The proportion of correctly predicted samples among all actual samples of a particular class.
- F1-Score: The harmonic mean of Precision and Recall, providing a balanced measure of classification performance.
- Support: The total number of actual samples belonging to each class.

**Misclassified Images**
The model also visualizes misclassified images, where the predicted label differs from the true label. Analyzing these images helps identify classes that are visually similar and provides insights into the model's limitations and areas for improvement.

## Code :

```
import tensorflow as tf

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense

from tensorflow.keras.optimizers import Adam

import matplotlib.pyplot as plt

import pandas as pd

(x_train,y_train),(x_test,y_test)=tf.keras.datasets.fashion_mnist.load_data()

x_train=x_train.reshape(-1,784)/255.0

x_test=x_test.reshape(-1,784)/255.0

def create_model(lr):

m=Sequential([Dense(128,activation='relu',input_shape=(784,)),

Dense(64,activation='relu'),

Dense(10,activation='softmax')])

m.compile(optimizer=Adam(learning_rate=lr),

loss='sparse_categorical_crossentropy',

metrics=['accuracy'])

return m
```

```
learning_rates=[0.1,0.01,0.001]
results=[]
for lr in learning_rates:
model=create_model(lr)
model.fit(x_train,y_train,epochs=5,batch_size=128,verbose=0)
loss,acc=model.evaluate(x_test,y_test,verbose=0)
results.append([lr,acc])
df=pd.DataFrame(results,columns=['Learning Rate','Accuracy'])
print(df)
plt.plot(df['Learning Rate'],df['Accuracy'],marker='o')
plt.xscale('log'); plt.grid(); plt.show()


epochs_list=[5,10,20]
res=[]
for ep in epochs_list:
model=create_model(0.001)
h=model.fit(x_train,y_train,validation_split=0.2,epochs=ep,batch_size=128,verbose=0)
loss,acc=model.evaluate(x_test,y_test,verbose=0)
res.append([ep,acc])
edf=pd.DataFrame(res,columns=['Epochs','Accuracy'])
print(edf)
plt.plot(edf['Epochs'],edf['Accuracy'],marker='o')
plt.grid(); plt.show()
model=create_model(0.001)
history=model.fit(x_train,y_train,validation_split=0.2,epochs=20,batch_size=128)
plt.plot(history.history['accuracy']); plt.plot(history.history['val_accuracy']); plt.legend(['Train','Val']); plt.show()
```

```
plt.plot(history.history['loss']); plt.plot(history.history['val_loss']); plt.legend(['Train','Val']); plt.show()

print(model.evaluate(x_test,y_test))

from sklearn.metrics import confusion_matrix, classification_report


import numpy as np

import seaborn as sns

import matplotlib.pyplot as plt

# Make predictions on the test set

y_pred_probs = model.predict(x_test)

y_pred = np.argmax(y_pred_probs, axis=1)

# Generate Confusion Matrix

cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(10, 8))

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')

plt.title('Confusion Matrix')

plt.xlabel('Predicted Label')

plt.ylabel('True Label')

plt.show()

print('\nClassification Report:')

print(classification_report(y_test, y_pred))


misclassified_idx = np.where(y_pred != y_test)[0]

class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',

'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']

# Plot a sample of misclassified images

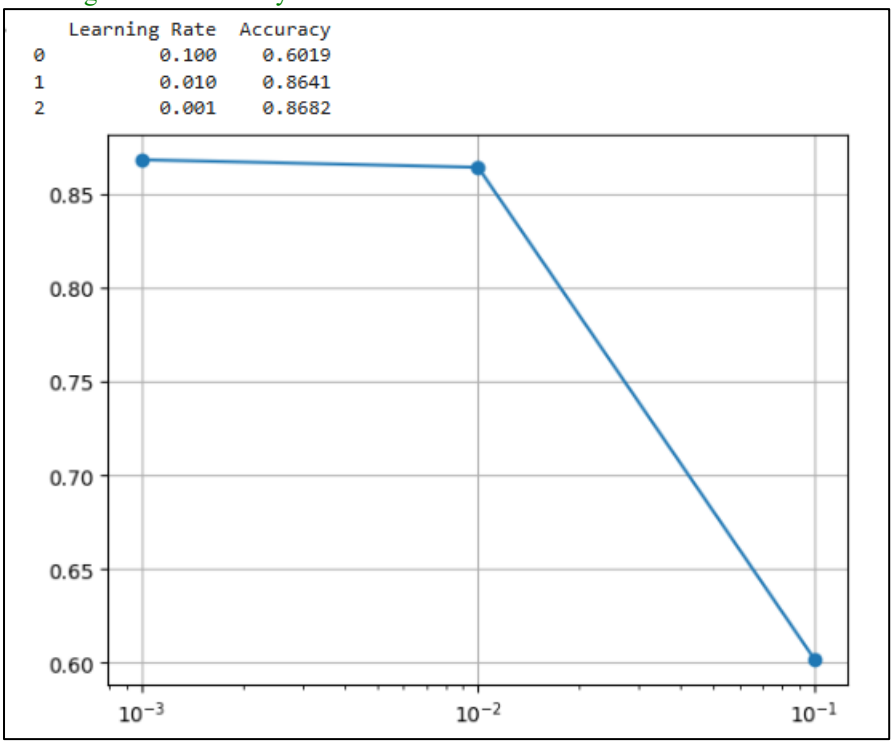
plt.figure(figsize=(15, 10))
```

```
for i, idx in enumerate(np.random.choice(misclassified_idx, size=25, replace=False)):
plt.subplot(5, 5, i + 1)
plt.imshow(x_test[idx].reshape(28, 28), cmap='gray')
plt.title(f"True: {class_names[y_test[idx]]}\nPred: {class_names[y_pred[idx]]}",
color='red' if y_pred[idx] != y_test[idx] else 'black')
plt.axis('off')
plt.suptitle('Misclassified Images (True Label vs. Predicted Label)', y=1.02, fontsize=16)
plt.tight_layout()
plt.show()
```

# Output :

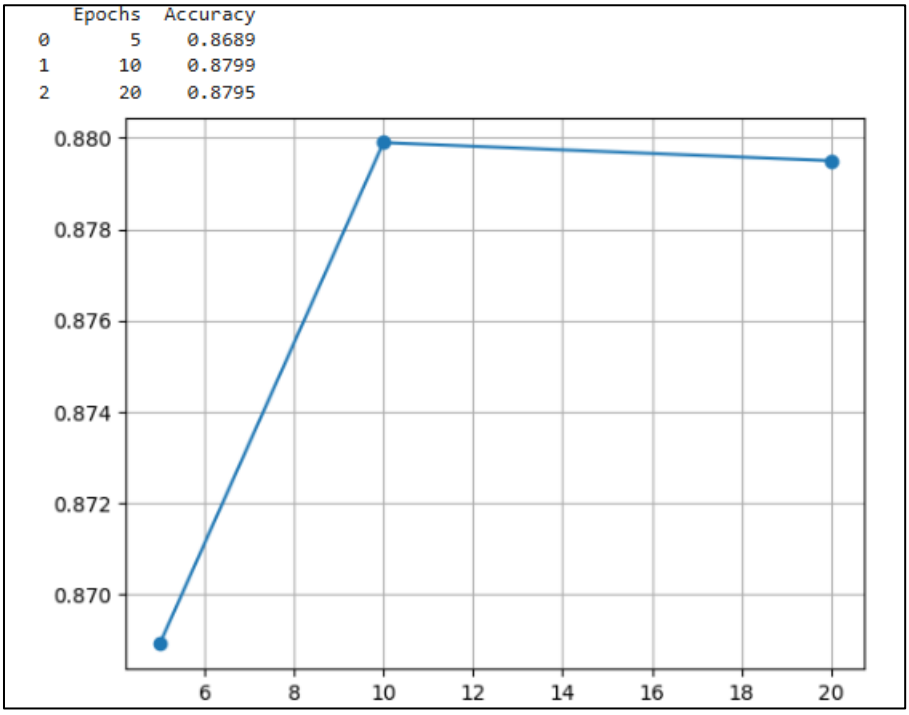
# Visualisation of data:

1. Learning rate Vs Accuracy



2. Epochs Vs Accuracy

| | Epochs | Accuracy |
|---|---|---|
| 0 | 5 | 0.8689 |
| 1 | 10 | 0.8799 |
| 2 | 20 | 0.8795 |

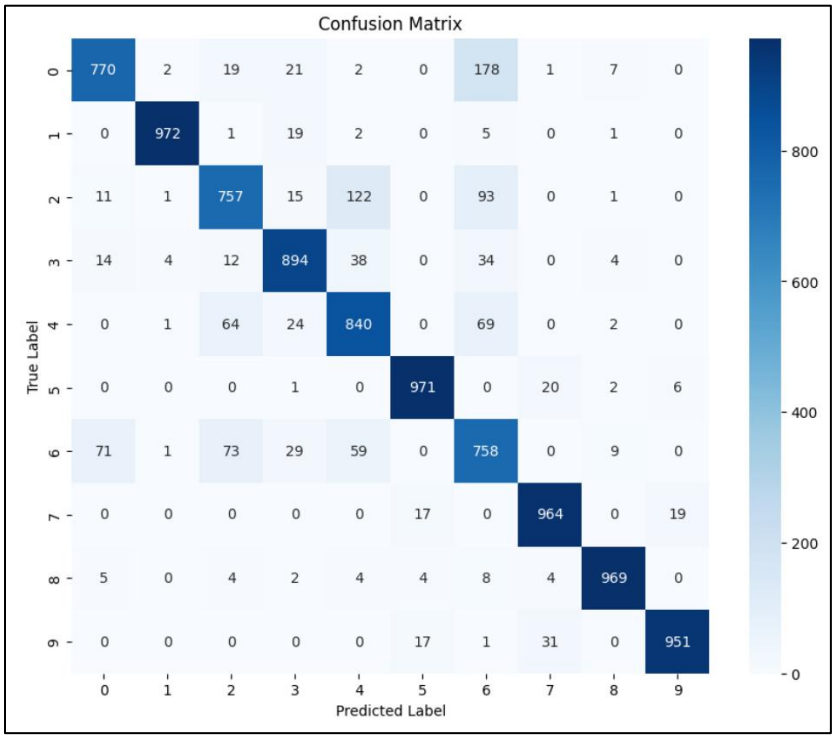



3. Effect on Accuracy and loss of training and testing data as no. of epoch increases

```
Epoch 1/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 4s 8ms/step - accuracy: 0.7957 - loss: 0.5832 - val_accuracy: 0.8444 - val_loss: 0.4380
Epoch 2/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - accuracy: 0.8558 - loss: 0.4080 - val_accuracy: 0.8578 - val_loss: 0.3987
Epoch 3/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.8689 - loss: 0.3682 - val_accuracy: 0.8698 - val_loss: 0.3654
Epoch 4/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - accuracy: 0.8773 - loss: 0.3399 - val_accuracy: 0.8777 - val_loss: 0.3517
Epoch 5/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.8840 - loss: 0.3185 - val_accuracy: 0.8809 - val_loss: 0.3312
Epoch 6/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.8897 - loss: 0.3039 - val_accuracy: 0.8657 - val_loss: 0.3663
Epoch 7/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 4s 9ms/step - accuracy: 0.8953 - loss: 0.2858 - val_accuracy: 0.8782 - val_loss: 0.3401
Epoch 8/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.8989 - loss: 0.2761 - val_accuracy: 0.8812 - val_loss: 0.3274
Epoch 9/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.9024 - loss: 0.2638 - val_accuracy: 0.8823 - val_loss: 0.3250
Epoch 10/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.9068 - loss: 0.2527 - val_accuracy: 0.8867 - val_loss: 0.3183
Epoch 11/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.9101 - loss: 0.2462 - val_accuracy: 0.8855 - val_loss: 0.3252
Epoch 12/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - accuracy: 0.9109 - loss: 0.2389 - val_accuracy: 0.8886 - val_loss: 0.3133
Epoch 13/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - accuracy: 0.9134 - loss: 0.2292 - val_accuracy: 0.8852 - val_loss: 0.3251
Epoch 14/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.9153 - loss: 0.2246 - val_accuracy: 0.8928 - val_loss: 0.3079
Epoch 15/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.9185 - loss: 0.2184 - val_accuracy: 0.8839 - val_loss: 0.3339
Epoch 16/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.9214 - loss: 0.2135 - val_accuracy: 0.8866 - val_loss: 0.3309
Epoch 17/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - accuracy: 0.9239 - loss: 0.2034 - val_accuracy: 0.8920 - val_loss: 0.3103
Epoch 18/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - accuracy: 0.9260 - loss: 0.1973 - val_accuracy: 0.8921 - val_loss: 0.3198
Epoch 19/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.9273 - loss: 0.1945 - val_accuracy: 0.8907 - val_loss: 0.3164
Epoch 20/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.9301 - loss: 0.1872 - val_accuracy: 0.8903 - val_loss: 0.3196
```




4. Classification Report and confusion matrix



Classification Report:

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| 0 | 0.88 | 0.77 | 0.82 | 1000 |
| 1 | 0.99 | 0.97 | 0.98 | 1000 |
| 2 | 0.81 | 0.76 | 0.78 | 1000 |
| 3 | 0.89 | 0.89 | 0.89 | 1000 |
| 4 | 0.79 | 0.84 | 0.81 | 1000 |
| 5 | 0.96 | 0.97 | 0.97 | 1000 |
| 6 | 0.66 | 0.76 | 0.71 | 1000 |
| 7 | 0.95 | 0.96 | 0.95 | 1000 |
| 8 | 0.97 | 0.97 | 0.97 | 1000 |
| 9 | 0.97 | 0.95 | 0.96 | 1000 |
| accuracy | | | 0.88 | 10000 |
| macro avg | 0.89 | 0.88 | 0.89 | 10000 |
| weighted avg | 0.89 | 0.88 | 0.89 | 10000 |

## 5. Visualizing Misclassified Images

